

Homework 3

Due date: Thursday, October 15, 2026, 11:59 PM on Gradescope

Overview:

In this assignment, you will write a single cycle-accurate simulator for a 32-bit RISC-V processor in Python. Your simulator reads a program and an initial memory image from two text files, then executes the program one instruction per cycle, exactly like a real single-cycle datapath: fetch, decode, execute, memory access, write back. You will not write any assembly. Instead, you will write the components that execute assembly: the decoder that parses a 32-bit instruction word, the ALU, the memory system, and the register file.

What You'll Practice:

- Decoding fixed-width instruction formats and extracting bit fields
- Sign extension and two's-complement arithmetic under 32-bit wraparound
- Control flow: conditional branches, jumps, and link registers
- Python bit manipulation (&, |, ^, <<, >>) on a fixed-width word

What Is Provided:

A working skeleton that plumbs in the sequence for command-line, run-loop, output-writers, and several constructors (memory/register files).

You will need to incorporate: `InsMem.readInstr`, `DataMem.readDataMem`, `DataMem.writeDataMem`, `RegisterFile.readRF`, `RegisterFile.writeRF`, and `SingleCycleCore.step`.

Setup:

1. Get your repository from Classroom 50: Link and sign in with your GitHub account
 - a. Accept the assignment on Classroom50 here:
<https://classroom50.org/cmpe120/cmpe-120-fa26/assignments/hw3/accept>
 - b. Open the repository in your editor of choice
2. Follow the README to understand how to run and test your CPU simulator.

Implementation Notes:

Both `imem.txt` and `dmem.txt` hold one byte per line, as eight characters of 0 and 1. Four consecutive lines are one 32-bit word, most significant byte first, and both memories are byte-addressable:

00000000

00000000

00100000
10000011

Those four lines are byte addresses 0, 1, 2 and 3, and together they form the 32-bit instruction at byte address 0. Every instruction is 4 bytes, so a program running straight through visits addresses 0, 4, 8, 12, ...

Data memory is 1000 bytes (MEM_SIZE). A dmem.txt shorter than that is zero-filled: every byte past the end of the file is 0.

Concretely, if dmem.txt begins 00000001, 00000010, 00000011, 00000100, then LW R1, R0, #0 loads 0x01020304, the first line is the most significant byte. Storing 0x01020304 back to address 0 must reproduce those four lines in that order. Byte order is the most common near-miss failure, so test it early. Every test case accesses memory only at multiples of 4.

Registers: 32 registers, x0 through x31, 32 bits wide, two's complement.

x0 is always set to 0. Reads the unsigned 32-bit value 0 ... 4294967295. All arithmetic wraps modulo 2^{32} .

Cycles: One instruction completes per cycle. Cycle n fetches and executes the instruction at whatever the program counter holds, then updates the program counter.

The Instruction Set:

Implement these 16 instructions.

Mnemonic	Type	Meaning
ADD	R	$rd = rs1 + rs2$
SUB	R	$rd = rs1 - rs2$
XOR	R	$rd = rs1 \wedge rs2$
OR	R	$rd = rs1 \mid rs2$
AND	R	$rd = rs1 \& rs2$
ADDI	I	$rd = rs1 + \text{sign_ext}(\text{imm})$
XORI	I	$rd = rs1 \wedge \text{sign_ext}(\text{imm})$

ORI	I	$rd = rs1 \mid \text{sign_ext}(imm)$
ANDI	I	$rd = rs1 \& \text{sign_ext}(imm)$
LW	I	$rd = \text{mem32}[rs1 + \text{sign_ext}(imm)]$
SW	S	$\text{mem32}[rs1 + \text{sign_ext}(imm)] = rs2$
BEQ	B	if $rs1 == rs2$: $pc += \text{sign_ext}(imm)$ else $pc += 4$
BNE	B	if $rs1 \neq rs2$: $pc += \text{sign_ext}(imm)$ else $pc += 4$
JAL	J	$rd = pc + 4$; $pc += \text{sign_ext}(imm)$
JALR	I	$rd = pc + 4$; $pc = (rs1 + \text{sign_ext}(imm)) \& \sim 1$
HALT	-	stop the simulation

Key:

R = Register

I = Immediate

S = Store

B = Branch

J = Jump

Note: $\text{mem32}[a]$ is the 32-bit big-endian word at address a . Branch and jump offsets are relative to the address of the branch itself, not $pc + 4$. JAL/JALR write $pc + 4$ into rd unless rd is $x0$, which gives a plain unconditional jump.

Bit fields and encodings:

Bit 0 is the least significant. Every instruction is 32 bits and puts its six fields in the same places:

31..25 24..20 19..15 14..12 11..7 6..0
 funct7 rs2 rs1 funct3 rd opcode

The register fields never move. When a format has no second source or no destination, those bits hold part of its immediate instead.

Format	Instructions	Immediate Location
R	ADD SUB XOR OR AND	N/A
I	ADDI XORI ORI ANDI LW JALR	[31:20]
S	SW	[31:25]=imm[11:5], [11:7]=imm[4:0]
B	BEQ BNE	[31]=imm[12], [30:25]=imm[10:5], [11:8]=imm[4:1], [7]=imm[11]
J	JAL	[31]=imm[20], [30:21]=imm[10:1], [20]=imm[11], [19:12]=imm[19:12]

The immediate is 12 bits in I and S, 13 in B and 21 in J, and is sign-extended from its own top bit (bit 11, 11, 12 and 20 respectively). For B and J bit 0 is always 0, because targets are even, so only 12 of the 13 and 20 of the 21 bits are stored. “Sign-extend from bit 11” therefore always means bit 11 of the immediate, not bit 11 of the instruction word.

The opcode, funct3 and funct7 values are the standard RV32I ones. Decode opcode first, then funct3, plus funct7 for R-type, which is what separates ADD from SUB when they share an opcode and a funct3. They are tabulated in The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, in the RV32I base instruction set (starts on page 27), available from <https://drive.google.com/file/d/1IswwIoyZn-Tnzj2OEXM9sPrm-DVHCvll/view?usp=sharing>.

You can check your decoding against the public programs: each tests/<case>/input/code.asm lists the address, encoding and disassembly of every instruction, and between them the six cases use all 16.

Note: HALT is 0xFFFFFFFF: detect it by the whole word, or by bits [6:0] == 111111.

Worked example:

imem.txt begins 00000000 00000000 00100000 10000011, i.e. 0x00002083. Opcode 00000111 is a load and funct3 010 makes it LW; rs1 is 00000 (x0), rd is 00001 (x1) and the immediate is 0. That is LW x1, 0(x0) → load the word at address 0 into x1.

Formatting Guidelines:

Write these four files into the --iodir directory when the program halts. The writers in the skeleton do this already; use them and the formats will be correct by construction.

File 1: RFResult.txt →

The 32 registers after the program halts, x0 first, one per line, each exactly 32 characters of 0 and 1:

```
00000000000000000000000000000000
0000000000000000000000000100100
```

File 2: DMEMResult.txt →

All 1000 bytes of data memory, byte address 0 first, one per line, each exactly 8 characters:

```
00000000
00000001
```

File 3: TraceResult.txt →

One line per cycle, in execution order. Lines beginning with # are comments, so the skeleton's header is ignored.

```
# cycle pc instr next_pc
0 0 0x06400093 4
1 4 0x02500113 8
```

Note: The columns are the cycle number (from 0); the byte address fetched during that cycle; the instruction word as 0x plus 8 lowercase hex digits; and the program counter fetched next. The last line is the cycle that executed HALT, for which next_pc equals pc.

File 4: PerformanceMetrics.txt →

```
Total Cycles: 17
Instructions Executed: 16
CPI: 1.062500
IPC: 0.941176
```

Note: What does matter is the values and the number of values: RFRResult.txt needs exactly 32 numbers, DMEMResult.txt exactly 1000, and TraceResult.txt exactly one row per cycle. Total Cycles and Instructions Executed must be exact; CPI and IPC are compared with about 0.5% tolerance.

Gradescope Submission:

There are 20 test cases, each a RISC-V program with its expected output.

6 are public and live in your repository, so you can see both the program and its expected output. 14 are hidden, used only by the autograder; they cover the same instructions and kinds of programs, so a correct simulator passes them without ever seeing them.

input/code.asm gives the address, encoding, disassembly and source of every instruction. When a case fails, it is the fastest way to see whether the problem is in your decoder, your control flow, or your memory access.

Scoring is out of 100 points: 20 cases $\times$ 5 points, and within a case each of the four output files is 1.25 points. A case earns full marks only if all four files match; per-file partial credit is reported, so you can see whether your registers, memory, trace or metrics are the problem.

What the autograder runs. It searches your submission for main.py and runs with a 10 second limit per test case; a program that does not halt in time fails that case. So keep the file named main.py, do not change the --iodir argument or the output file names, do not modify the provided writers, and do not expect anything else of yours to be run. Anything else you leave in the repository is ignored.

On Gradescope, open the assignment, choose GitHub as the submission method, authenticate with the account that owns your repository, and select the repository and branch containing main.py. If Gradescope cannot download it, download the repository as a zip and upload that instead. Downloading instructions are here: <https://docs.github.com/en/repositories/working-with-files/using-files/downloading-files-from-github>. You will see a per-test-case breakdown.

Troubleshooting:

- NotImplementedError: InsMem.readInstr, you have not filled in the TODO methods yet.

- Registers right but DMEMResult.txt wrong, byte order. In tests/03_load_store the program stores 0x01234567 at byte address 32, so bytes 32–35 of your output must read 00000001, 00100011, 01000101, 01100111.
- Right cycles but wrong next_pc, offsets are relative to the branch instruction: pc + immediate, not pc + 4 + immediate.
- A loop never terminates, the counter is not updating, or you are comparing sign-extended values against unsigned ones. Compare the 32-bit patterns directly.
- A negative immediate behaves as a large positive number, you are not sign-extending. ADDI R2, R0, #-1 must give 0xFFFFFFFF.
- The program runs past HALT, or a length mismatch, set halt=True for the HALT cycle and stop (instructions after HALT must never execute); RFResult.txt needs exactly 32 lines and DMEMResult.txt exactly 1000.
- Too slow, use plain lists and integers, do not rebuild the memory image on every access, and build output with join rather than repeated append.

Bit Manipulation In Python:

Most of this assignment is pulling fields out of a 32-bit word and putting them back together. Use these patterns and you will not fight Python's integer semantics.

Reading a field, shift down, then mask to the field width:

```
opcode = instr & 0x7F          # bits [6:0]
rd      = (instr >> 7) & 0x1F  # bits [11:7]
```

Sign extension, a 12-bit 0xFFF means -1, not 4095. Write one helper and reuse it:

```
sign = 1 << (bits - 1)
return (value & (sign - 1)) - (value & sign)
```

Formatting values back into text:

```
format(value, "032b")    # 32-character binary, for a register
format(byte, "08b")      # 8-character binary, for a memory byte
format(instr, "08x")     # 8 lowercase hex digits, for the trace
```

Note: Mask to 32 bits when you store a value (value & 0xFFFFFFFF), and sign-extend when an immediate is meant to be negative. Also note >> is an arithmetic shift in Python, so -1 >> 1 is -1, not 2147483647.

References:

Official tutorial: <https://docs.python.org/3/tutorial/>

Standard types: <https://docs.python.org/3/library/stdtypes.html>

Bitwise operations reference:

<https://docs.python.org/3/reference/expressions.html#binary-bitwise-operations>

A practical walkthrough: <https://realpython.com/python-bitwise-operators/>

Venv: <https://docs.python.org/3/library/venv.html>

Conda environments:

<https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>

Note: If you are coming from C or Java: Python uses and/or (not &&/||) and capitalised True/False; integer division is // and floors towards negative infinity; and there is no fixed-width integer type, which is why you must mask with & 0xFFFFFFFF. for i in range(n): excludes the upper bound.